



LeetCode 2169: Count Operations to Obtain Zero

1. Problem Title & Link

Title: LeetCode 2169: Count Operations to Obtain Zero

Link: <https://leetcode.com/problems/count-operations-to-obtain-zero/>

2. Problem Statement (Short Summary)

You are given two non-negative integers:

num1

num2

Perform the following operation repeatedly:

- If $\text{num1} \geq \text{num2}$, then:

$\text{num1} = \text{num1} - \text{num2}$

- Otherwise:

$\text{num2} = \text{num2} - \text{num1}$

Continue until one of the numbers becomes:

0

Return the total number of operations performed.

Example

Input:

num1 = 2

num2 = 3

Process:

2,3

$3 = 3 - 2 = 1$

2,1

$2 = 2 - 1 = 1$

1,1

$1 = 1 - 1 = 0$



0,1

Operations:

3

Output:

3

3. Examples (Input → Output)

Example 1

Input:

num1 = 2

num2 = 3

Output:

3

Example 2

Input:

num1 = 10

num2 = 10

Output:

1

Example 3

Input:

num1 = 5

num2 = 4

Output:

5

4. Constraints

$0 \leq \text{num1}, \text{num2} \leq 10^5$

Important:

Stop immediately when either number becomes 0.

5. Core Concept (Pattern / Topic)



Simulation

Secondary Topics:

- Repeated Subtraction
- Euclidean Algorithm Idea

6. Thought Process (Step-by-Step Explanation)

Understanding the Rule

At every step:

Subtract the smaller number from the larger number.

Example

Input:

num1 = 5

num2 = 4

Step 1:

$5 - 4 = 1$

1,4

Step 2:

$4 - 1 = 3$

1,3

Step 3:

$3 - 1 = 2$

1,2

Step 4:

$2 - 1 = 1$

1,1

Step 5:

$1 - 1 = 0$



0,1

Operations:

5

Brute Force Observation

The problem literally tells us what to do.

So we simply simulate.

Key Insight

Continue until:

`num1 == 0`

or

`num2 == 0`

Count every operation.

Connection to GCD

Notice:

Larger -= Smaller

This resembles the Euclidean Algorithm used for finding GCD.

However, this problem only asks for operation count.

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

5 , 4

Step 1:

1 , 4

Step 2:

1 , 3

Step 3:

1 , 2

Step 4:

1 , 1

Step 5:

0 , 1

Stop.

Answer:

5



8. Pseudocode (Language Independent)

```
count = 0

while num1 != 0 and num2 != 0

    if num1 >= num2

        num1 -= num2

    else

        num2 -= num1

    count++

return count
```

9. Code Implementation

Python

```
class Solution:
    def countOperations(
        self,
        num1: int,
        num2: int
    ) -> int:

        count = 0

        while num1 != 0 and num2 != 0:

            if num1 >= num2:
                num1 -= num2
            else:
                num2 -= num1

            count += 1

        return count
```

Java

```
class Solution {

    public int countOperations(
        int num1,
        int num2) {
```



```
int count = 0;

while(num1 != 0 &&
      num2 != 0) {

    if(num1 >= num2)
        num1 -= num2;
    else
        num2 -= num1;

    count++;
}

return count;
}
```

10. Time & Space Complexity

Time Complexity

Worst case:

$O(\max(\text{num1}, \text{num2}))$

because repeated subtraction may happen many times.

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

while num1 != 0 or num2 != 0

Wrong.

Should be:

while num1 != 0 and num2 != 0

Mistake 2

Forgetting to increment count.

Mistake 3

Continuing after one number becomes zero.

Problem says stop immediately.

**Edge Cases****num1 = 0**

Input:

0,5

Output:

0

No operation needed.

Equal Numbers

Input:

10,10

Operation:

 $10 - 10 = 0$

Answer:

1

One Number Is One

Input:

1,5

Operations:

 $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

Answer:

5

12. Detailed Dry Run

Input:

num1 = 2

num2 = 3

Operation	num 1	num 2
Start	2	3
1	2	1
2	1	1
3	0	1

Stop.



Return:

3

13. Common Use Cases (Real-Life / Interview)

Resource Reduction

Repeatedly consume larger resource using smaller resource.

Mathematical Simulations

Step-by-step state updates.

Euclidean Algorithm Foundation

Understanding GCD logic.

Interview Purpose

Tests:

Simulation

Loops

Condition Handling

14. Common Traps (Important!)

Trap 1

Overthinking the problem.

The intended solution is direct simulation.

Trap 2

Using division instead of subtraction.

The problem explicitly says:

Subtract

Trap 3

Incorrect loop condition.

Need:

Both numbers non-zero

15. Builds To (Related LeetCode Problems)

LC 2169 - Count Operations to Obtain Zero





LC 1342 - Number of Steps to Reduce Number to Zero



LC 202 - Happy Number



LC 263 - Ugly Number



GCD / Euclidean Algorithm

These strengthen simulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Simulation	$O(\max(n, m))$	$O(1)$	Best
Optimized Euclidean Style	$O(\log n)$	$O(1)$	Advanced

Approach 1: Direct Simulation (Chosen)

Matches problem statement exactly.

Easy to understand.

Approach 2: Optimized Idea

Instead of subtracting one-by-one:

`count += num1 // num2`

Similar to GCD optimization.

More advanced than required.

17. Why This Solution Works (Short Intuition)

At each step, one number decreases while both remain non-negative.

Eventually one number becomes zero.

Counting every subtraction operation exactly matches the process described in the problem.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the final pair.

Example:

`(0, gcd)`

**Follow-Up 2**

Return the GCD.

Leads directly to:

Euclidean Algorithm

Follow-Up 3

Optimize using division.

Advanced version.

Follow-Up 4

Track the sequence of states.

Example:

(5,4)

(1,4)

(1,3)

(1,2)

(1,1)

(0,1)

Interview Takeaway

The key pattern is:

Simulation

Whenever you hear:

Perform operation repeatedly

Follow exact rules

Update state until condition met

think:

Simulation

Many easy interview problems are solved simply by implementing the process exactly as described, without searching for a complex algorithm.